

A Hybrid Rule-Based and LLM-Assisted Framework for Natural Language-to-SQL Generation with Semantic Caching

Hemavathy R.

*Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
hemavathy@rvce.edu.in*

Rohith Arsha

*Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
rrohitarsha.scs25@rvce.edu.in*

Dhruva K

*Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
dhruvak.scs25@rvce.edu.in*

Abstract—Natural Language-to-SQL (NL2SQL) systems have gained significant attention for enabling users to interact with relational databases using natural language instead of Structured Query Language (SQL). Recent Large Language Model (LLM)-based approaches have substantially improved SQL generation accuracy; however, they continue to suffer from repeated model inference, increased response latency, high computational cost, and occasional schema inconsistencies. To address these challenges, this paper presents SemanticSQL, a hybrid Natural Language-to-SQL framework that integrates deterministic rule-based query generation, LLM-assisted SQL synthesis, semantic caching, and schema-aware validation within a unified architecture. The proposed framework first attempts deterministic SQL generation using predefined rules, followed by semantic similarity search over a Redis-backed cache to reuse previously generated SQL queries for semantically similar requests. When neither approach is applicable, the system invokes a locally hosted Ollama Large Language Model to generate SQL, which is subsequently validated against the underlying database schema before execution. The framework also incorporates runtime model management, configurable semantic similarity thresholds, query analytics, and database administration modules to improve usability and extensibility. Experimental evaluation demonstrates that semantic caching effectively reduces redundant LLM inference and improves query response efficiency while maintaining reliable SQL generation. The modular architecture enables seamless integration of additional language models, database systems, and optimization techniques, making SemanticSQL a scalable and practical solution for intelligent natural language database interaction.

Index Terms—Natural Language-to-SQL (NL2SQL); Large Language Models (LLMs); Semantic Caching; Redis; Ollama; Schema Validation; Hybrid Query Generation; Database Management Systems; Natural Language Processing; Text-to-SQL.

I. INTRODUCTION

Relational databases form the backbone of modern information systems across domains such as healthcare, finance,

education, and enterprise resource planning. Although Structured Query Language (SQL) remains the standard language for retrieving and manipulating relational data, constructing SQL queries requires knowledge of database schemas and query syntax, making database interaction challenging for non-technical users. Natural Language-to-SQL (NL2SQL) systems address this limitation by enabling users to express database queries in natural language, thereby improving accessibility and simplifying data retrieval.

Recent advances in Large Language Models (LLMs) have significantly improved the performance of NL2SQL systems by leveraging their strong natural language understanding and reasoning capabilities. Modern approaches employ prompt engineering, schema linking, fine-tuning, and retrieval-augmented techniques to generate SQL queries with higher execution accuracy. Despite these improvements, existing LLM-based NL2SQL systems continue to face several challenges. Repeated execution of semantically similar queries results in redundant LLM inference, increasing response latency and computational cost. Furthermore, generated SQL may contain schema inconsistencies or invalid table and column references, reducing execution reliability. Most existing systems also provide limited support for runtime model management, configurable execution parameters, and comprehensive query analytics.

To address these limitations, this paper proposes **SemanticSQL**, a hybrid Natural Language-to-SQL framework that combines deterministic rule-based query generation with LLM-assisted SQL synthesis, Redis-backed semantic caching, and schema-aware SQL validation. Instead of invoking the language model for every user query, the proposed framework first performs deterministic rule matching, followed by semantic similarity search over previously executed queries. If no suitable cached result exists, a locally hosted Ollama

Large Language Model generates the SQL statement, which is subsequently validated against the underlying database schema before execution. This hybrid processing pipeline reduces unnecessary LLM inference while maintaining accurate and reliable SQL generation.

In addition to hybrid query processing, SemanticSQL incorporates runtime model management, configurable semantic similarity thresholds, query history, semantic cache analytics, database exploration, and schema management within a unified web-based platform. The modular architecture enables seamless integration of additional language models, embedding techniques, caching strategies, and relational database systems, making the framework suitable for both research and practical deployment.

The primary contributions of this work are summarized as follows:

- A hybrid Natural Language-to-SQL framework that combines deterministic rule-based query generation with Large Language Model-assisted SQL synthesis.
- A Redis-backed semantic caching mechanism that reduces redundant LLM inference through semantic similarity matching.
- A schema-aware SQL validation module that improves execution reliability by detecting invalid schema references before query execution.
- Runtime model management and configurable semantic similarity thresholds that enable dynamic system configuration without application restart.
- A modular web-based platform integrating semantic caching, database exploration, query analytics, and system monitoring to support intelligent database interaction.

The remainder of this paper is organized as follows. Section II reviews recent research on Natural Language-to-SQL systems, semantic caching, and Large Language Models. Section III presents the proposed SemanticSQL framework and its architecture. Section IV describes the experimental setup and implementation details. Section V discusses the experimental results and performance analysis. Finally, Section VI concludes the paper and outlines future research directions.

II. RELATED WORK

Natural Language-to-SQL (NL2SQL) has witnessed significant progress with the emergence of Large Language Models (LLMs), enabling users to interact with relational databases using natural language instead of Structured Query Language (SQL). Recent research has primarily focused on improving SQL generation accuracy through semantic parsing, schema understanding, prompt engineering, and fine-tuning of language models.

Semantic caching has recently emerged as an effective approach for reducing redundant LLM inference and improving response latency. Li *et al.* proposed SCALM, a semantic caching architecture that exploits semantic similarity among user queries to improve cache utilization and reduce token consumption in LLM-based applications [1]. Their work demonstrated substantial improvements in cache hit ratio and

operational efficiency, highlighting the importance of semantic caching for large-scale language model deployments. However, SCALM focuses primarily on conversational AI systems rather than Natural Language-to-SQL applications.

Several studies have focused on improving SQL generation using Large Language Models. Grounding Natural Language to SQL Translation with Data-Based Self-Explanations improves SQL generation by incorporating database content into the reasoning process, thereby reducing hallucinations and improving execution correctness [2]. MetaSQL introduces a generate-then-rank framework that produces multiple candidate SQL queries and selects the most appropriate query through ranking, resulting in improved execution accuracy [3]. Performance evaluations of open-source LLMs further demonstrate that locally deployable models such as Llama, Qwen, Mixtral, and Gemma provide competitive performance for Text-to-SQL tasks while significantly reducing deployment costs [4]. Other studies have investigated intelligent database interfaces such as AskDB [5] and analyzed the trade-offs between reasoning and non-reasoning language models for SQL generation, highlighting the balance between inference quality, computational cost, and response latency [6].

Schema understanding remains one of the most critical challenges in Text-to-SQL systems because natural language expressions must be accurately mapped to database tables and columns. Recent studies have proposed several approaches to improve schema linking and prompt construction. Turkish schema-context integration demonstrates that enriching prompts with schema information improves SQL generation across multilingual environments [7]. Fine-tuning Large Language Models specifically for Text-to-SQL tasks has also shown improved execution accuracy by adapting models to database-specific query patterns [8]. Knowledge-guided frameworks such as KG-SQL incorporate external semantic knowledge to improve complex query generation [9], while Lexa-Query extends LLM-based SQL generation with user-friendly interfaces for intelligent database interaction [10]. Similar approaches have also been explored for NoSQL databases [11] and optimized language-model frameworks such as NL2SQL-OLM [12].

Domain-specific NL2SQL systems have further demonstrated the applicability of language models in specialized applications. A Natural Language-to-SQL framework for power communication planning enables efficient querying of engineering databases using natural language interfaces [13]. Phrase-based schema-linking techniques such as PS-SQL improve the mapping between natural language phrases and schema elements by leveraging phrase representations instead of individual tokens, resulting in improved domain generalization [14]. Similarly, RH-SQL introduces refined schema selection and hardness-aware prompting to reduce irrelevant schema information while improving SQL generation performance with lower computational overhead [15].

Although considerable progress has been achieved in SQL generation, schema linking, prompt engineering, and language model optimization, existing approaches typically address

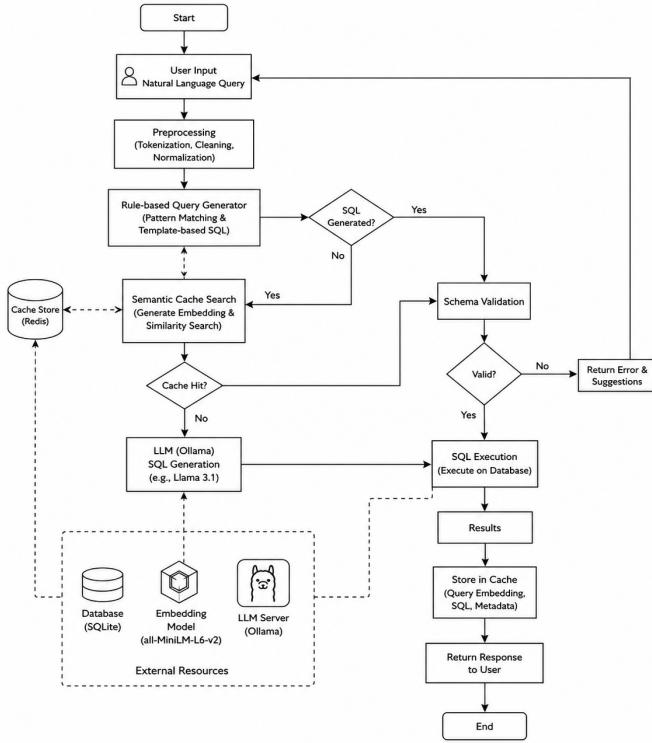


Fig. 1. Overall workflow of the proposed SemanticSQL framework.

these components independently. Limited research integrates deterministic rule-based query generation, semantic caching, schema-aware SQL validation, runtime model management, configurable similarity thresholds, and query analytics within a unified framework. Motivated by these observations, the proposed **SemanticSQL** framework combines hybrid rule-based and LLM-assisted SQL generation with Redis-backed semantic caching, schema validation, runtime configuration, and analytics to provide an efficient, extensible, and practical Natural Language-to-SQL system

III. PROPOSED METHODOLOGY

The proposed SemanticSQL framework combines deterministic rule-based query generation, semantic caching, Large Language Model (LLM)-assisted SQL generation, and schema-aware validation within a unified architecture. The objective is to minimize unnecessary LLM inference while maintaining reliable and efficient Natural Language-to-SQL (NL2SQL) translation. Instead of relying solely on language models for every user query, the framework adopts a hierarchical processing strategy that prioritizes deterministic query generation and semantic cache retrieval before invoking the LLM. Figure 1 illustrates the overall workflow of the proposed system.

A. System Overview

SemanticSQL follows a multi-stage query processing pipeline. Initially, the user's natural language query is pre-

processed and analyzed by a rule-based query generator. If the query matches one of the predefined deterministic patterns, the corresponding SQL statement is generated directly without requiring language model inference. Queries that cannot be resolved through deterministic rules are subsequently searched within a semantic cache using vector similarity search. If a semantically similar query exists in the cache, the previously generated SQL statement is reused. Otherwise, the query is forwarded to a locally deployed Large Language Model through Ollama for SQL generation.

IV. EXPERIMENTAL SETUP

This section describes the implementation environment, software configuration, database setup, and evaluation methodology adopted to validate the proposed SemanticSQL framework. The experiments were designed to evaluate the effectiveness of the hybrid query generation pipeline, semantic caching mechanism, schema-aware validation, and runtime configuration capabilities.

A. Development Environment

SemanticSQL was implemented as a full-stack web application consisting of a React-based frontend and a FastAPI backend. The frontend provides an interactive interface for natural language query submission, query history visualization, cache monitoring, and runtime configuration. The backend is responsible for query processing, SQL generation, semantic caching, schema validation, and database interaction.

The application utilizes SQLite as the relational database management system and Redis as the semantic cache store. SQLAlchemy is employed as the Object Relational Mapper (ORM) for database communication, while Ollama provides local deployment of Large Language Models for SQL generation. Sentence embeddings are generated using the all-MiniLM-L6-v2 model from the Sentence-Transformers library.

B. System Configuration

The experimental environment was configured using Python 3.11 with FastAPI as the backend framework and React with Vite for frontend development. Redis was deployed to maintain semantic cache entries and runtime configuration parameters. Ollama was configured to execute locally hosted language models for SQL generation, thereby eliminating dependence on external cloud APIs and reducing inference costs.

The modular system architecture allows individual components, including the language model, embedding model, database, and cache, to be replaced or upgraded independently without affecting the overall framework.

C. Database Setup

The evaluation was performed on a relational database containing representative tables commonly used in enterprise database applications. The database includes entities such as employees, departments, products, customers, orders, and categories. These tables were populated with sample records to evaluate a variety of Natural Language-to-SQL queries

involving selection, filtering, aggregation, sorting, grouping, and join operations.

The deterministic rule engine was evaluated using predefined query templates, while more complex natural language queries were processed through the Large Language Model pipeline. The generated SQL statements were validated against the database schema before execution to ensure syntactic and semantic correctness.

D. Evaluation Methodology

The proposed framework was evaluated based on multiple functional and performance aspects. Initially, incoming natural language queries were processed by the deterministic rule engine. Queries that could not be resolved through predefined rules were compared against cached query embeddings using cosine similarity. Cache misses triggered SQL generation through the configured Large Language Model, followed by schema-aware validation and SQL execution.

The evaluation focused on assessing the effectiveness of the hybrid query generation strategy, semantic cache retrieval, schema validation, runtime model management, and overall query execution workflow. Particular emphasis was placed on reducing redundant LLM inference through semantic caching while maintaining reliable SQL generation and execution across different categories of natural language queries.

V. RESULTS AND DISCUSSION

The proposed SemanticSQL framework was successfully implemented and evaluated on a relational database containing multiple entities, including employees, products, customers, orders, and categories. The system effectively translated natural language queries into executable SQL statements using a hybrid query generation pipeline consisting of deterministic rules, semantic caching, and Large Language Model (LLM)-assisted SQL generation.

A. Functional Evaluation

The deterministic rule engine successfully generated SQL queries for predefined query patterns without requiring language model inference, resulting in low-latency query execution. Queries that did not match any predefined rule were processed using semantic similarity search over cached query embeddings. When semantically similar queries were identified, previously generated SQL statements were retrieved directly from the Redis cache, eliminating redundant LLM inference.

For cache misses, the framework invoked the configured Ollama language model to generate SQL dynamically. The generated SQL statements were subsequently validated against the database schema before execution, reducing failures caused by invalid table or column references. This hybrid execution strategy enabled reliable SQL generation across both simple and complex natural language queries.

B. Semantic Cache Performance

The semantic cache effectively reduced repeated language model invocations by reusing SQL generated for semantically equivalent queries. Runtime modification of the semantic similarity threshold enabled administrators to control cache sensitivity without restarting the application. Experimental observations indicated that lowering the similarity threshold increased cache hits, whereas higher thresholds improved semantic precision by reducing false-positive matches.

The Redis-backed implementation also provided persistent cache storage, query statistics, and runtime monitoring capabilities, allowing efficient management of cached query embeddings and generated SQL statements.

C. System Functionality

The developed framework integrates several modules within a unified web interface, including:

- Natural Language-to-SQL query interface
- Deterministic rule-based SQL generation
- LLM-assisted SQL generation using Ollama
- Redis-backed semantic caching
- Schema-aware SQL validation
- Runtime model management
- Configurable semantic similarity threshold
- Database exploration and schema visualization
- Query history and cache analytics

The modular architecture enables seamless integration of additional embedding models, language models, database systems, and query optimization techniques without requiring major architectural modifications.

D. Discussion

The proposed SemanticSQL framework demonstrates that combining deterministic query generation with semantic caching and LLM-assisted SQL synthesis provides an efficient solution for Natural Language-to-SQL translation. Deterministic rules ensure fast execution for frequently occurring query patterns, while semantic caching minimizes redundant inference for semantically similar queries. The incorporation of schema-aware validation further improves execution reliability by preventing invalid SQL statements from reaching the database.

Compared with conventional LLM-only approaches, the hybrid architecture improves computational efficiency, enhances execution reliability, and provides greater flexibility through runtime configuration and modular system design. These characteristics make SemanticSQL suitable for intelligent database interaction in both academic and practical deployment environments.

VI. CONCLUSION AND FUTURE WORK

This paper presented **SemanticSQL**, a hybrid Natural Language-to-SQL framework that combines deterministic rule-based query generation, semantic caching, Large Language Model (LLM)-assisted SQL synthesis, and schema-aware validation within a unified architecture. By prioritizing

deterministic query processing and semantic cache retrieval before invoking the language model, the proposed framework effectively reduces redundant LLM inference while maintaining reliable SQL generation. The integration of Redis-backed semantic caching, runtime model management, configurable similarity thresholds, and schema validation further enhances system efficiency, flexibility, and execution reliability.

The implemented framework demonstrates that combining traditional rule-based techniques with modern LLM capabilities provides an effective balance between performance and adaptability. Deterministic rules enable low-latency execution for frequently occurring query patterns, while semantic caching minimizes repeated computation for semantically similar queries. The modular architecture also supports seamless integration of different embedding models, language models, relational database systems, and administrative functionalities, making SemanticSQL suitable for both research and practical deployment.

Although the proposed framework provides an efficient and extensible solution for Natural Language-to-SQL translation, several opportunities remain for future research. The current rule engine relies on manually defined deterministic rules, which may become difficult to maintain as database schemas and query complexity increase. Future work will focus on developing intelligent rule generation techniques capable of automatically learning and adapting rules for newly introduced database schemas using machine learning and Large Language Models. Additional research directions include adaptive semantic similarity threshold optimization, multi-database support, advanced schema linking, query optimization, support for distributed database systems, and comprehensive benchmarking across multiple open-source and proprietary language models. These enhancements are expected to further improve the scalability, robustness, and practical applicability of SemanticSQL for intelligent database querying.

REFERENCES

- [1] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu, "SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models," in *Proc. IEEE/ACM 32nd Int. Symp. on Quality of Service (IWQoS)*, 2024.
- [2] Y. Wang *et al.*, "Grounding Natural Language to SQL Translation with Data-Based Self-Explanations," 2024.
- [3] X. Zhang *et al.*, "MetaSQL: A Generate-Then-Rank Framework for Natural Language to SQL Translation," 2024.
- [4] M. Ahmed *et al.*, "Performance Evaluation of Open-Source Large Language Models for Text-to-SQL Conversion in Healthcare Data," in *IEEE EMBS International Conference on Biomedical and Health Informatics (BHI)*, 2025.
- [5] A. Kumar *et al.*, "AskDB: An Intelligent Natural Language Querying System for Relational Databases," 2024.
- [6] S. Patel *et al.*, "Evaluating the Cost Trade-offs of Reasoning and Non-Reasoning Large Language Models in Text-to-SQL Tasks," 2025.
- [7] H. Demir *et al.*, "Enhancing Text-to-SQL Conversion in Turkish Through Schema Context Integration," 2025.
- [8] L. Chen *et al.*, "Fine-Tuning Large Language Models for Text-to-SQL Generation," 2024.
- [9] K. Zhao *et al.*, "KG-SQL: Knowledge-Guided Text-to-SQL Generation Using Large Language Models," 2024.
- [10] R. Singh *et al.*, "LexaQuery: A Large Language Model-Based Natural Language Interface for SQL Query Generation," 2025.

- [11] J. Smith *et al.*, "Natural Language Query System for NoSQL Databases Using Large Language Models," 2025.
- [12] P. Roy *et al.*, "NL2SQL-OLM: An Optimized Language Model Framework for Natural Language to SQL Generation," 2025.
- [13] X. Wang *et al.*, "Power Communication Planning Data Query System Based on the NL2SQL Framework," in *Proc. IEEE Int. Conf.*, 2025.
- [14] Z. Lan and S. Li, "PS-SQL: Phrase-based Schema-Linking with Pre-trained Language Models for Text-to-SQL Parsing," in *Proc. 6th Int. Conf. on Natural Language Processing (ICNLP)*, IEEE, 2024.
- [15] J. Yi, G. Chen, and Z. Shen, "RH-SQL: Refined Schema and Hardness Prompt for Text-to-SQL," in *Proc. 6th Int. Conf. on Electronic Engineering and Informatics (EEI)*, IEEE, 2024.